

2.2.2 Of Rainbows and Salt

Once you have the password hash, then what? How can you get a user's password from a given hash? How can you exploit hashes to compromise security? Most importantly, how can you secure against such attacks.

To reiterate the way password hashes work, when you login, it is your password that is used to authenticate you, however instead of using your password to authenticate, it uses the hash of the password.

Example:

User's selected password: qb7%R1e

Stored password hash (MD5) : 4f55ffad7d8c44c2538ccaf5b5ef407

User provides the password: h6Sk3ty

Hash for password: a444724e17a3a3a5704a6fb81db4dd60

Authentication fails

User provides the password : qb7%R1e

Hash for password : 4f55ffad7d8c44c2538ccaf5b5ef407

Authentication succeeds

A very simple way of recovering a password from a hash would be to apply the same techniques we do to password cracking. This is, to generate possible password candidates, hash them, and see if the hash matches. As you can imagine, this will be a time consuming process, although since this process will be offline we will not have to worry about limited retry attempts.

One advantage we have with hashes though is that one can easily create a database of plaintexts and their corresponding hashes. This way, in the future one can simply look up the hash in their database and they will have the password instantaneously instead of needing to wait hours!

By creating an exhaustive database of string-hash pairs one would need to calculate the hashes only once, and in the future this database could be utilized to look up a string from the corresponding hash.

Unfortunately, creating such a database of hashes will require an incredible amount of storage space, and will only work up to a point. A 10 character password as discussed before has 9510 combinations (additionally, smaller passwords will also need to be tested) which is a rather large number (59873693923837890625). Even storing a list of all possible 10-character passwords will take 544548074 TB of space! With the hashes, the storage requirement will become 1415824992 TB. While these kinds of storage capacities are not technically impossible, they are currently not feasible.

Even so, it might be feasible to have a database containing hashes of all dictionary words, and variations thereof. However, dictionary attacks are already much faster than brute-force attacks, and this just means that the